The 99% Real-World Architecture Study Guide

Welcome, Cloud Architect

You already understand containers, Kubernetes, CI/CD, and Terraform. Now it's time to map those concepts to AWS. We are skipping the noise (sorry, Amazon Ground Station) and focusing purely on the core services used in 99% of production workloads.

🌍 Global Infrastructure

AWS isn't just "the cloud"; it's physical hardware distributed globally.

- **Regions:** A geographic area (e.g., `us-east-1` in N. Virginia). Always pick a region close to your users to reduce latency.
- **Availability Zones (AZs):** Distinct data centers within a Region, connected by low-latency links (e.g., `us-east-1a`, `us-east-1b`). *Rule of thumb: Always deploy across at least two AZs for High Availability.*
- **Edge Locations:** Mini-datacenters globally distributed used purely for caching (CloudFront) and DNS (Route 53) to serve content fast.

🤝 Shared Responsibility Model

Security is a partnership. If a hacker gets in, whose fault is it?

- **AWS is responsible for security OF the cloud:** Hardware, physical facilities, hypervisors, and the network infrastructure.
- **You are responsible for security IN the cloud:** Operating system patches, firewall rules (Security Groups), IAM roles, and customer data.

> ⚠️ **Warning:** If you leave an S3 bucket public and data leaks, that's on you (Security IN the cloud). If an AWS hard drive fails and they didn't shred it properly, that's on them.

🗺️ The DevOps Learning Roadmap

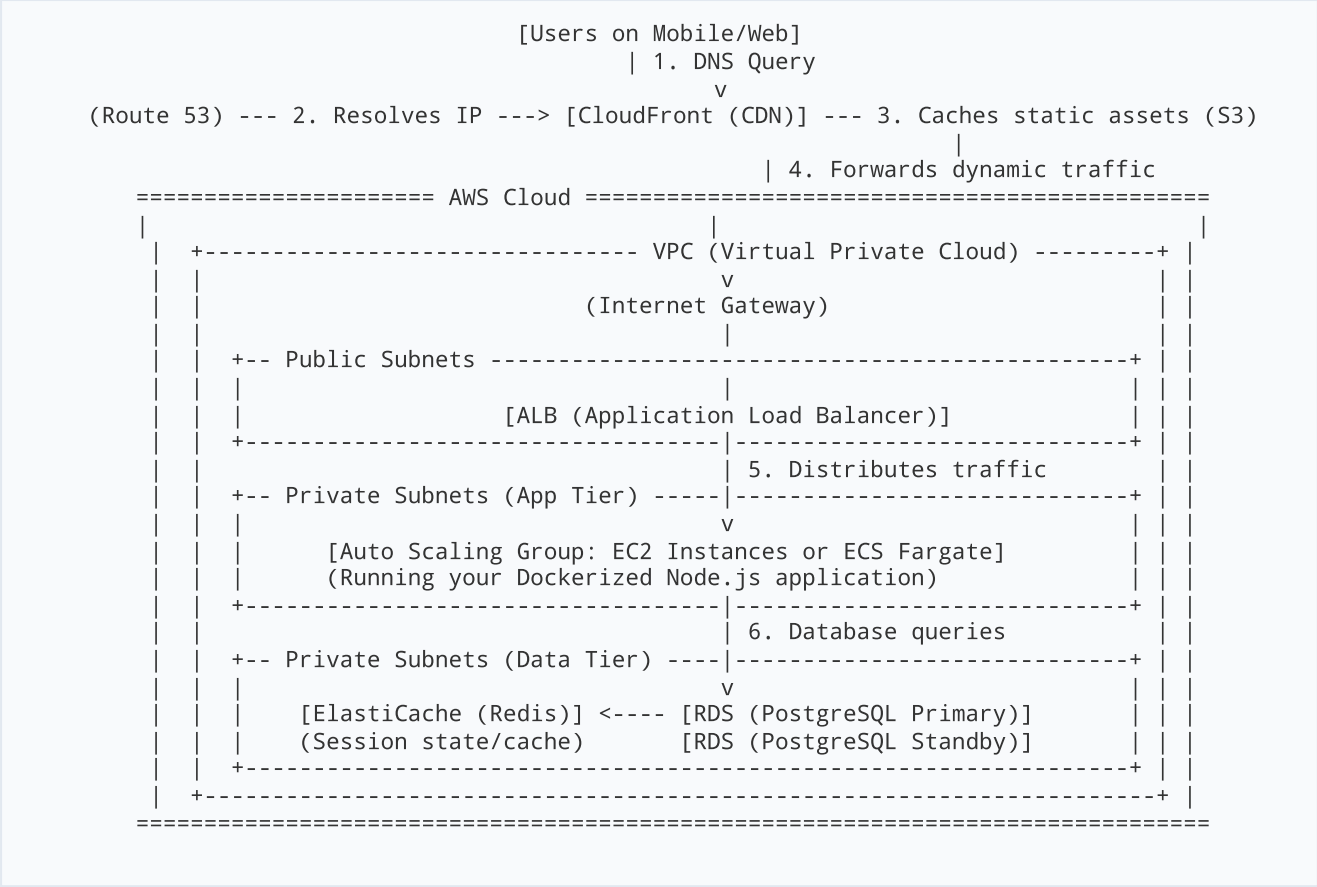
Don't learn alphabetically. Learn architecturally. Services are tagged by priority:

Tier 1: Core Foundation (Must Know)	Tier 2: Production Readiness (Should Know)
Compute: EC2, Lambda **Networking:** VPC, Security Groups, Route 53, ALB **Storage:** S3, EBS **Identity/Security:** IAM, Secrets Manager **Databases:** RDS, DynamoDB	**Containers:** ECS, EKS, Fargate **Messaging:** SQS, SNS, EventBridge **Monitoring:** CloudWatch, CloudTrail **Caching/CDN:** ElastiCache, CloudFront **Cost:** Cost Explorer

> 📝 **Note:** Since you know Terraform, skip AWS CloudFormation initially. Focus on using Terraform to provision these "Must Know" resources.

🖼️ The Big Picture: 3-Tier Architecture

Before diving into individual services, let's look at how they connect. A modern, highly available web application (like a Node.js/React stack) deployed on AWS typically follows this flow. *(Search: "AWS 3-Tier Architecture Diagram" for official visuals).*



How the traffic flows:

1. **Route 53 (DNS):** The user types `app.com`. Route 53 acts as the phonebook, translating the URL to an AWS endpoint.
2. **CloudFront & S3 (CDN/Storage):** CloudFront intercepts the request. If the user wants a static image, CloudFront serves it instantly from its edge cache (pulled originally from an S3 bucket).
3. **ALB (Load Balancing):** If it's an API request, traffic enters the VPC via the Internet Gateway and hits the Application Load Balancer in the Public Subnet.
4. **EC2/ECS (Compute):** The ALB routes the request to a healthy worker node in a Private Subnet. If traffic spikes, Auto Scaling spins up more nodes.
5. **RDS/ElastiCache (Database):** The worker node queries ElastiCache for fast, temporary data. If not found, it queries the RDS relational database. The RDS Standby ensures no downtime if the primary fails.

> 💡 **Tip:** Notice the **IAM** and **CloudWatch** services aren't in the flow? They wrap the entire architecture. IAM controls *who* can provision these resources, and CloudWatch monitors the *metrics and logs* of all of them.

Category 1: Compute ⚙️

Compute is where your code actually executes. As a DevOps engineer, you'll choose between Virtual Machines (EC2), Containers (ECS/EKS), or Serverless functions (Lambda).

💻 Amazon EC2 (Elastic Compute Cloud) MUST KNOW

What it is: Resizable virtual servers in the cloud where you run applications, install software, and have full OS-level control.

Analogy: Renting a physical computer in someone else's data center by the second, instead of buying one.

Why and when to use it: Legacy monolithic apps, custom databases, stateful workloads, or when you need explicit control over the OS and kernel.

Key concepts:

- **AMI (Amazon Machine Image):** The template (OS + pre-installed software) used to boot the instance.
- **Instance Type:** The hardware combo (e.g., `t3.micro` is cheap/burstable, `c5.large` is compute-optimized).
- **User Data:** A bash script that runs exactly *once* during the first boot (great for bootstrapping).

Pricing model:

Billed per second. Three main ways to buy:

- **On-Demand:** Pay as you go (most expensive, most flexible).
- **Reserved:** Commit to 1-3 years (up to 72% discount).
- **Spot:** Bid on spare capacity (up to 90% discount, but AWS can terminate it with 2-mins notice).

Best practices & pitfalls: Don't treat EC2s like pets; treat them like cattle. Don't SSH in to fix things—update your Terraform/Ansible and redeploy. *Pitfall:* Leaving large unneeded instances running over the weekend.

> 🎯 **Exam Focus:** Know your instance families! **C** = Compute, **R** = RAM (Memory), **M** = Multipurpose/General, **T** = Tiny/Burstable.

📈 Amazon Auto Scaling MUST KNOW

What it is: A service that automatically adds or removes compute resources (like EC2 instances) based on real-time traffic or CPU load.

Analogy: A restaurant manager who calls in extra waiters when a big bus of tourists arrives, and sends them home when it's empty.

How it works:

⚖️ ELB (Elastic Load Balancing - ALB/NLB) MUST KNOW

What it is: A managed service that automatically distributes incoming application traffic across multiple targets (EC2, containers, IP addresses).

Analogy: The traffic cop at a busy intersection directing cars to lanes with the least traffic.

Why and when to use it: To ensure high availability and prevent any single server from becoming overwhelmed. Used as the single point of entry for your web apps.

Key concepts:

- **ALB (Application Load Balancer):** Layer 7 (HTTP/HTTPS). Smart routing based on URLs (e.g., `/api` goes to server A, `/images` to server B).
- **NLB (Network Load Balancer):** Layer 4 (TCP/UDP). Ultra-high performance, millions of requests per second. Handles raw network traffic.
- **Target Group:** The logical group of servers the load balancer is sending traffic to.

Common example: Terminating SSL/TLS certificates at the ALB so your backend EC2 instances don't have to spend CPU cycles decrypting HTTPS traffic.

Best practices & pitfalls: Always enable "Cross-Zone Load Balancing" so traffic is evenly split across availability zones. *Pitfall:* Forgetting to allow health checks in your Security Group, causing the ELB to think all instances are dead.

⚡ AWS Lambda MUST KNOW

What it is: A Serverless compute service that lets you run code without provisioning or managing servers.

Analogy: A vending machine. It costs you nothing while it sits there. When someone pushes a button (event), it dispenses a snack (runs code), you pay a tiny fee, and it shuts off.

Why and when to use it: Event-driven tasks (cron jobs, file processing), microservices, building APIs. Perfect for unpredictable traffic where you don't want idle servers.

How it works: You upload a ZIP file of your code (Python, Node, Go) or a Docker image. AWS provisions a micro-container, runs the function, returns the result, and destroys the container.

Pricing model: Pay per request AND per millisecond of compute time. 1 million free requests per month.

> 💡 **Best practices & pitfalls:** Keep your deployment packages small for fast startup. Beware of **Cold Starts** (the delay when an idle function is triggered for the first time).

> 🎯 **Exam Focus:** Lambda has a maximum execution time limit of **15 minutes**. If your job takes

Containers on AWS 🐳

Since you know Docker and Kubernetes, this section is crucial. AWS splits container management into two parts: the **Control Plane** (orchestration) and the **Data Plane** (where containers actually run).

📦 Amazon ECS (Elastic Container Service) SHOULD KNOW

What it is: AWS's proprietary, highly integrated container orchestration service.

Analogy: Kubernetes' simpler, easier-to-manage cousin who only speaks "AWS".

Why and when to use it: When you want to run Docker containers in production but don't want the immense overhead of managing Kubernetes clusters. It integrates natively with ALB, IAM, and CloudWatch.

Key concepts:

- **Task Definition:** The blueprint (like a Pod spec or Docker Compose file).
- **Task:** A running instance of a Task Definition (like a Pod).
- **Service:** Ensures 'N' number of tasks are always running and attaches them to a Load Balancer (like a K8s Deployment).

☸️ Amazon EKS (Elastic Kubernetes Service) SHOULD KNOW

What it is: A managed Kubernetes service where AWS runs the K8s control plane (API server, etcd) for you.

Analogy: Renting a fully-staffed factory management team, but you still have to bring the factory floor workers.

Why and when to use it: When your company already heavily uses Kubernetes, requires multi-cloud portability, or relies on the rich open-source K8s ecosystem (Helm, Istio, ArgoCD).

Pricing model: $0.10 per hour just for the control plane (~$72/month) PLUS the cost of the underlying compute nodes.

> ⚠️ **Pitfall:** EKS networking (AWS VPC CNI) can be complex. Every Pod gets a real VPC IP address, which means you can easily exhaust your subnet's IP addresses!

👻 AWS Fargate MUST KNOW

What it is: A serverless compute engine (Data Plane) for containers that works with both ECS and EKS.

🌱 AWS Elastic Beanstalk GOOD TO KNOW

What it is: A Platform-as-a-Service (PaaS) that automatically handles deployment, provisioning, load balancing, and auto-scaling.

Analogy: Heroku, but built on top of AWS.

Why and when to use it: For developers who don't know AWS and just want to upload code (Java, Node, PHP) and have it run. *As a DevOps engineer, you will rarely use this* because you want the control of Terraform + ECS/EKS.

Pricing model: Free itself; you only pay for the EC2/ALB resources it creates under the hood.

⚖️ Compute Comparisons

In interviews and architecture design, you must know how to choose the right tool.

EC2 vs. Lambda vs. Fargate

Feature	EC2 (Virtual Machine)	Fargate (Serverless Container)	Lambda (Serverless Function)
Control	Full OS access. You manage everything.	Container level. No OS access.	Code level. Only bring code.
Start Time	Minutes (booting OS)	Seconds (pulling image)	Milliseconds
Max Duration	Unlimited	Unlimited	15 Minutes Maximum
Best for...	Legacy apps, specific kernel tweaks.	Microservices, Dockerized apps.	Event triggers, cron jobs, simple APIs.

ECS vs. EKS

Feature	Amazon ECS	Amazon EKS
Complexity	Low. Very easy to learn and integrate.	High. Requires Kubernetes knowledge.
Ecosystem	AWS proprietary. Tightly integrated with AWS.	Open-source Kubernetes ecosystem.
Control Plane Cost	Free (if using EC2/Fargate data plane).	~$72/month per cluster control plane.
When to choose?	"I just want to run Docker on AWS easily."	"We are migrating from on-prem K8s to